

Evolving Modular Neural Networks

Charlotte Gibson¹, Marcus Ghosh^{1,2}

¹ Department of Electrical and Electronic Engineering, Imperial College London, United Kingdom.

² I-X Centre for AI in Science, Imperial College London, United Kingdom.

September 20, 2026

Abstract

Evolving neural architectures that capture modularity and recurring patterns remains challenging due to the vast search spaces of standard neuroevolution methods. We present an evolutionary algorithm (EA) that constrains this space by drawing from structural banks of modules or motifs, evolving their selection and connectivity. Structural banks support knowledge-free or knowledge-driven search and may contain diverse module types. By reducing the search space, the EA achieves faster convergence than traditional methods while producing more interpretable results. Distributional analysis reveals emergent structure–function relationships aligned with task demands.

1 Introduction

A longstanding objective of Neuroevolution has been to discover neural architectures with modularity, symmetry, and recurring motifs—properties that support efficient computation, adaptability, and biological plausibility. Yet evolving such structures remains difficult.

Direct encodings such as NEAT [1] face rapidly expanding search spaces as networks grow, making the emergence of structured solutions unlikely without strong bias. Indirect encodings, such as HyperNEAT [2], can promote regularity but often fail when irregular structures are required. This highlights the need for algorithms that can flexibly discover both regular and irregular structures according to task demands. ExNEAT [3] introduced the idea of evolving a single module and duplicating it according to a user-specified layout. While extremely efficient for spatially symmetric tasks, it required strong prior knowledge to set module size and duplication rules. Together, these approaches highlight the need for algorithms that can flexibly discover both regular and irregular structures depending on task demands.

We address this challenge with an EA that assembles networks from modules or motifs drawn from predefined structural banks. The algorithm evolves the selection and connectivity of these components while their internal structures remain fixed. This design allows prior knowledge to shrink the search space, or generic banks to support open-ended discovery.

Structural banks can contain any component compatible with the network: simple units or small circuits, larger architectures such as CNNs or transformers, or biologically inspired motifs. Tracking distributions across generations provides insight into structure–function relationships and reveals emergent properties such as symmetry and diversity.

We evaluate the EA on image classification and multimodal maze tasks. In both domains it converges more rapidly than NEAT [1] and finds solutions where NEAT [1] failed within a reasonable number of generations. Distributional analysis

further confirms task-aligned properties, illustrating both the efficiency and interpretability of the approach.

2 Method

The EA follows a standard evolutionary framework of population initialisation, evaluation, selection, and variation, but constrains the search space through structural banks. Each genome specifies a set of modules or motifs, and their connectivity to inputs and outputs. Evolution only acts on which components are selected from the bank and how they are wired into the network. Mutations include connection addition, removal, and swapping, weight modification, and module swapping. Crossover is implemented as one-point recombination.

Altering the bank contents determines the scope of the search: generic components enable knowledge-free exploration, while task-specific modules provide knowledge-driven search.

Two variants are implemented. The *module* version relies on user-defined components, which may range from simple units to complex architectures. The *motif* version generates all three-node circuits with chosen properties such as feedforward, recurrent, and temporal (non-feedforward), offering a biologically inspired search space. Both share the same evolutionary pipeline, differing only in how banks are constructed, and which distributional statistics are tracked.

Interpretability is supported through distributional analysis. By tracking the frequency of modules or motifs across generations, the algorithm reveals structure–function relationships and emergent properties such as symmetry and diversity. This analytic layer complements the efficiency gains over NEAT [1] by explaining how structures are exploited during evolution.

Algorithm 1 Elitist Evolutionary Algorithm with Structural Banks

Inputs: population size μ ; elitism k ($1 \leq k < \mu$); crossover and mutation rates (p_c, p_m); task $\tau \in \{\text{maze}, \text{image}\}$; structural bank $\mathbf{B}_\tau$; evaluation environment $\mathcal{E}_\tau$

Outputs: per-generation record of best genome, mean elite fitness, and module/motif distributions `trial_data`

```

1:  $P \leftarrow \text{INITIALIZEPOPULATION}(\mu, \mathbf{B}_\tau)$ 
2:  $g \leftarrow 0$ 
3: while not  $\text{TERMINATIONCONDITION}(g)$  do
4:   Evaluate: compute  $f(x) \leftarrow \text{EVALUATOR}(x \mid \mathcal{E}_\tau, \sigma, n_{\text{steps}}, \tau) \forall x \in P$   $\triangleright$  in parallel chunks
5:   Select:  $F \leftarrow \text{TOPK}(P, k)$   $\triangleright$  retain  $k$  fittest elites
6:   Record: trial_data[ $g$ ]  $\leftarrow$  best genome  $x^*$ , mean elite fitness, and module/motif distribution
7:   Reproduce:
8:      $n_{\text{new}} \leftarrow \mu - k$   $\triangleright$  offspring to generate
9:     Draw  $n_x \sim \text{Binomial}(n_{\text{new}}, \frac{p_c}{p_c + p_m})$ 
10:    Set  $n_{\text{mut}} = n_{\text{new}} - n_x$ 
11:    for  $i = 1$  to  $n_x$  do
12:      Select parents  $p_1, p_2 \in F$ 
13:      offspring  $\leftarrow \text{CROSSOVER}(p_1, p_2)$ 
14:    end for
15:    for  $i = 1$  to  $n_{\text{mut}}$  do
16:      Select parent  $p \in F$ 
17:      offspring  $\leftarrow \text{MUTATE}(\text{CLONE}(p))$ 
18:    end for
19:     $P \leftarrow F \cup O$   $\triangleright$  repopulate  $|P| = \mu$ 
20:     $g \leftarrow g + 1$   $\triangleright$  update generation
21: end while

```

3 Results

3.1 Track Maze Task

The track maze task (Figure 1) requires an agent to reach a goal in a linear maze using directional cues. Cues include both misleading and informative signals, with gaps that necessitate some memory. Agents sense cues at each step and act via a policy defined by the evolved network.

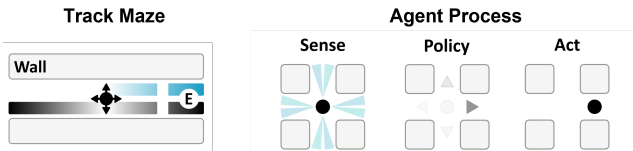


Figure 1: Track maze task.

This task was solved with the *motif* variant of the EA, using banks of all possible three-node motifs (Figure 2)

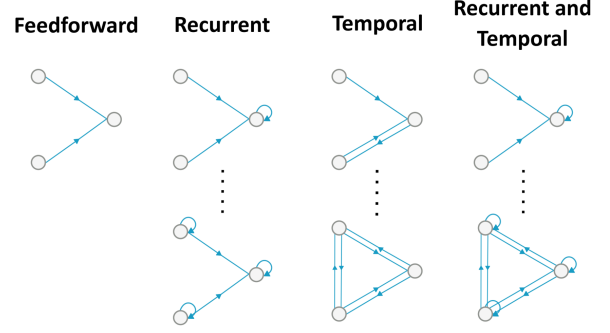


Figure 2: Motifs with Feedforward, Recurrent (.R), Temporal (.T), Recurrent and Temporal (.RT).

Results (Figure 3) show that NEAT [1] and the feed-forward-only motif bank fail to solve the task, whereas banks containing recurrent and/or temporal connections converge to a solution. Convergence speed does not simply follow bank size, but correlates with the proportion of motifs capable of solving the task. Distributional analysis confirms this: motifs types incapable of contributing to a solution (feedforward only) fall rapidly to zero, while motifs containing recurrent and/or recurrent structure dominate. The specific type that dominates varies stochastically across runs as.

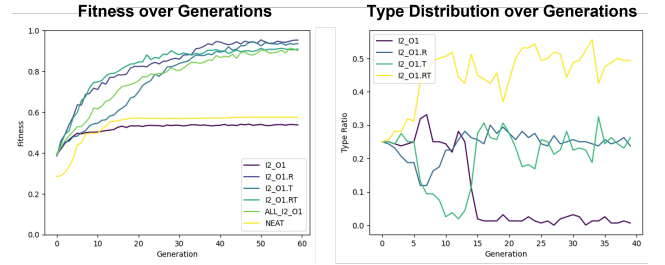


Figure 3: Track maze results.

3.2 Square Classification Image Task

The square classification task (Figure 4) requires identifying the center of a 3×3 square within a 9×9 image with an additional small 1×1 square. This task used the module variant of the EA with a bank containing (i) Bus, (ii) Convolution (3×3 kernel of all 1s), (iii) Argmax, and (iv) Feedforward.

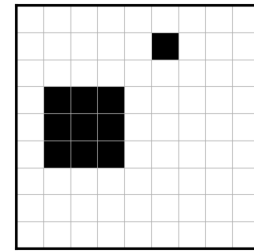


Figure 4: Square image classification task.

Results (Figure 5) show that NEAT [1] fails to find a solution, while the EA rapidly converges. Distributional analysis demonstrates that the feedforward that the feedforward

module, used only for unbiased initialisation, is quickly eliminated, and in the subsequent generations the remaining modules distribute evenly, reflecting their joint necessity for solving the task.

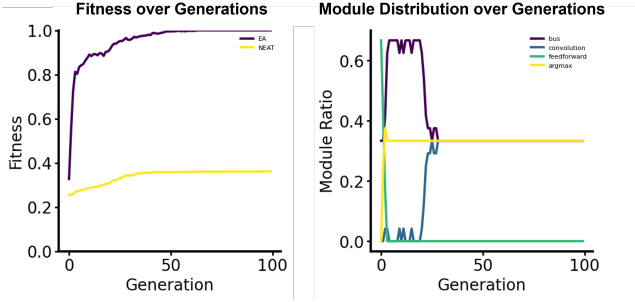


Figure 5: Square image classification results.

3.3 Cross Classification Image Task

The cross classification task (Figure 6) requires identifying the center of a 3×3 cross within a 9×9 image with an additional 3×3 plus. The module bank contained: (i) Bus, (ii) Correlation (3×3 kernel of alternating 1s and -1s), and (iv) Feedforward.

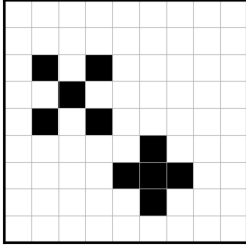


Figure 6: Cross image classification task.

Results (Figure 7) show that NEAT [1] again fails to solve the task whereas the EA converges. As in the square task, the feedforward module is eliminated early, and the remaining modules distribute evenly.

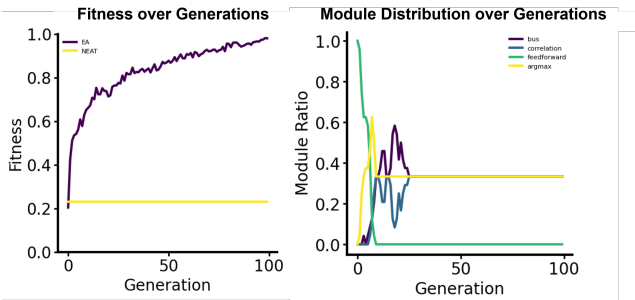


Figure 7: Cross image classification results.

4 Discussion and Conclusion

The proposed evolutionary algorithm demonstrates that constraining the search space with structural banks of modules or motifs provides both efficiency and interpretability. By evolving the selection and connectivity of predefined components, the method achieves faster convergence than NEAT [1]

and produces solutions that reveal task-aligned structural regularities. Tracking distributions of modules or motifs across generations offers a unique analytic layer, enabling the study of structure–function relationships and highlighting emergent properties such as symmetry and diversity. Compared to other method, the approach balances flexibility with control: users can inject prior knowledge through task-specific banks or retain open-ended discovery with generic banks.

Future work will extend evaluation to more complex and demanding domains, providing a stronger test of scalability. Beyond accuracy, we will investigate robustness and generalisation by training on tasks with one set of properties and testing on others, such as increased noise levels, cue sparsity, or altered dynamics. This will help assess whether evolved networks capture transferable structural principles rather than overfitting to specific conditions. Finally, by applying distributional analysis in these richer settings, we expect to uncover deeper structure–function relationships, yielding insight into how evolution exploits modularity, symmetry, and diversity to solve increasingly complex problems.

In summary, this evolutionary framework offers a flexible means of incorporating structural priors into neuroevolution, combining faster convergence with greater interpretability, and providing a foundation for systematic study of modularity in complex neural architectures.

References

- [1] K. O. Stanley and R. Miikkulainen, “Evolving neural networks through augmenting topologies,” *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.
- [2] K. O. Stanley, D. B. D’Ambrosio, and J. Gauci, “A hypercube-based encoding for evolving large-scale neural networks,” in *Artificial Life*, vol. 15, pp. 185–212, MIT Press, 2009.
- [3] M. Ghosh and M. Zhang, “Exneat - separable network through coevolution of partially-dependent sub-modules,” 2025.